

École Supérieure d'Informatique

Première Année du Cycle Préparatoire
Algorithmique et Structures de Données Dynamiques

Théorie des Ensembles sur Structures Linguistiques

Utilisation des Structures de Données Dynamiques en C

Réalisé par

ABDELMOUMENE Hamza Ayoub
TALBI Baghdad

Encadreur : Dr. Kermi

Année universitaire 2025–2026 | Date de rendu : 10 mai 2026

Contents

1	Introduction	2
2	Objectifs	2
3	Structures de Données et Justification	2
3.1	Hierarchie à trois niveaux	2
3.2	Arbre Binaire de Recherche (niveau mot)	3
3.2.1	Équilibrage sans AVL	3
3.3	Listes Chaînées (niveaux phrase et paragraphe)	4
3.3.1	Optimisation par tableau d'index	4
4	Machine Abstraite	4
4.1	Macros pour les listes chaînées	4
4.2	Macros pour l'ABR	5
5	Implémentation	5
5.1	Analyseur syntaxique (parser)	5
5.2	Opérations ensemblistes	5
5.3	Gestion de la mémoire	6
6	Interface Utilisateur	6
6.1	Architecture visuelle	6
6.2	Fonctionnalités clés	7
7	Tests et Validation	7
7.1	Fichiers de test	7
7.2	Cas limites traités	7
8	Conclusion	7

1 Introduction

La théorie des ensembles constitue l'une des branches les plus fondamentales des mathématiques. Un **ensemble** est une collection bien définie d'éléments distincts, non ordonnés. Les trois opérations principales, définies sur deux ensembles A et B d'un même univers U , sont :

$$A \cup B = \{x \mid x \in A \vee x \in B\} \quad A \cap B = \{x \mid x \in A \wedge x \in B\} \quad A \setminus B = \{x \mid x \in A \wedge x \notin B\}$$

Ce projet applique ces définitions formelles à des données linguistiques : chaque fichier texte est représenté comme une hiérarchie d'ensembles imbriqués, implémentée en C à l'aide de structures de données dynamiques. Chaque fichier est décomposé en paragraphes, chaque paragraphe en phrases, et chaque phrase en un ensemble de mots uniques normalisés.

2 Objectifs

L'objectif principal est de concevoir et implémenter, en langage C :

1. La lecture et le pré-traitement d'un ou plusieurs fichiers texte.
2. La décomposition de chaque fichier en une hiérarchie à trois niveaux (paragraphes $\rightarrow$ phrases $\rightarrow$ mots).
3. La représentation de chaque niveau par une structure de données dynamique adéquate.
4. L'implémentation des opérations ensemblistes aux trois niveaux.
5. Une interface utilisateur interactive en mode terminal.

3 Structures de Données et Justification

3.1 Hiérarchie à trois niveaux

Le contenu d'un fichier chargé est représenté par une structure imbriquée :

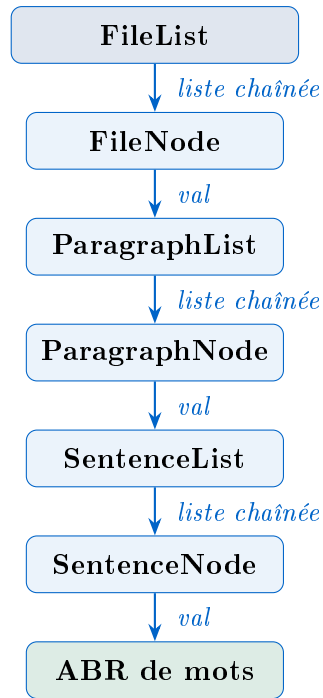


Figure 1: Hiérarchie des structures de données

3.2 Arbre Binaire de Recherche (niveau mot)

L'ABR est retenu pour représenter l'ensemble des mots d'une phrase pour les raisons suivantes :

- Le rejet des doublons est implicite : insérer un mot déjà présent ne fait rien (comportement ensembliste naturel).
- Le parcours *infixe* produit une liste triée alphabétiquement sans étape supplémentaire.
- L'insertion et la recherche sont en $O(\log n)$ lorsque l'arbre est équilibré.

3.2.1 Équilibrage sans AVL

Plutôt que des rotations AVL, l'implémentation utilise une **construction en deux passes** :

1. Collecter tous les mots dans un tableau dynamique `char**`.
2. Trier ce tableau alphabétiquement avec `qsort`.
3. Insérer par *médiane en premier* de façon récursive (`MedianInsert`).

Insérer la médiane d'un tableau trié en premier garantit une hauteur $\lfloor \log_2 n \rfloor$, identique à un arbre parfaitement équilibré, sans aucun code de rotation.

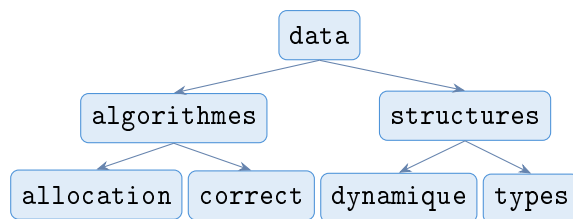


Figure 2: Exemple d'ABR construit par insertion médiane

3.3 Listes Chaînées (niveaux phrase et paragraphe)

Une liste chaînée simple avec pointeur de queue maintenu offre un ajout en $O(1)$, indispensable pendant l'analyse syntaxique où le nombre de phrases et de paragraphes est inconnu à l'avance.

3.3.1 Optimisation par tableau d'index

Après l'analyse, un parcours `BuildIndex` remplit un tableau `Node** index` en une passe : tous les accès aléatoires ultérieurs (sélection de menu, opérations ensemblistes) passent de $O(n)$ à $O(1)$.

Table 1: Comparaison des structures de données utilisées

Niveau	Structure	Insértion	Recherche	Justification
Mot	ABR équilibré	$O(\log n)$	$O(\log n)$	Unicité, tri naturel
Phrase	LLC + index	$O(1)$	$O(1)^*$	Taille dynamique
Paragraphe	LLC + index	$O(1)$	$O(1)^*$	Taille dynamique

* Après `BuildIndex` ; $O(n)$ avant.

4 Machine Abstraite

L'implémentation respecte strictement les conventions de la **machine abstraite** imposées par le cours, en utilisant des macros pour masquer les détails de la représentation physique.

4.1 Macros pour les listes chaînées

Table 2: Machine abstraite – liste chaînée

Macro	Expansion	Rôle
<code>Allouer(p)</code>	<code>(p) = CheckedMalloc(sizeof(*(p)), ...)</code>	Allouer un maillon
<code>Libérer(p)</code>	<code>free(p)</code>	Libérer un maillon
<code>Ass_val(p,v)</code>	<code>(p)->val = (v)</code>	Affecter le champ valeur
<code>Ass_adr(p,q)</code>	<code>(p)->addr = (q)</code>	Affecter le pointeur suivant
<code>Valeur(p)</code>	<code>(p)->val</code>	Lire la valeur
<code>Suivant(p)</code>	<code>(p)->addr</code>	Lire l'adresse suivante

4.2 Macros pour l'ABR

Table 3: Machine abstraite – arbre binaire de recherche

Macro	Expansion	Rôle
AllouerNoeud()	AllocateNodeImpl()	Allouer un nœud
LibérerNoeud(p)	free(p)	Libérer un nœud
Ass_FG(p,q)	(p)->left = (q)	Affecter le fils gauche
Ass_FD(p,q)	(p)->right = (q)	Affecter le fils droit
FG(p)	(p)->left	Lire le fils gauche
FD(p)	(p)->right	Lire le fils droit
Ass_Val_Noeud(p,v)	strcpy((p)->val, (v))	Stocker la chaîne
ValeurNoeud(p)	(p)->val	Lire la chaîne

5 Implémentation

5.1 Analyseur syntaxique (parser)

L'analyseur lit le fichier caractère par caractère en maintenant un état `ParserState` qui accumule le mot, la phrase et le paragraphe courants. La logique de dispatch est la suivante :

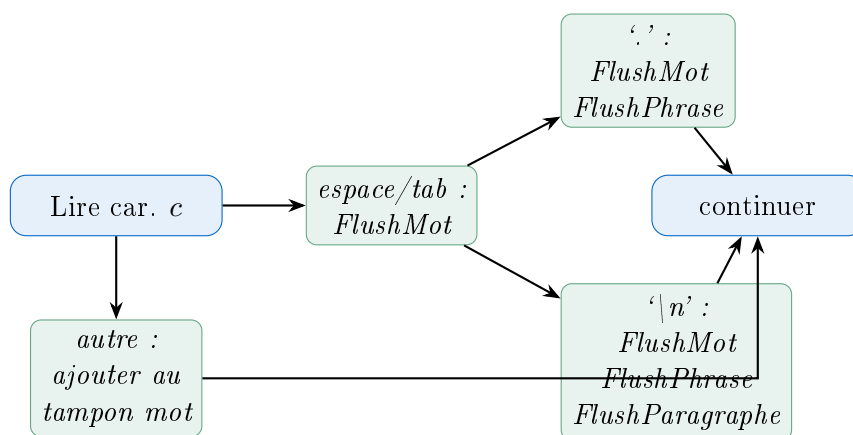


Figure 3: Logique de dispatch de l'analyseur

La construction en deux passes garantit l'équilibrage de chaque ABR de phrase : **(1)** collecte des mots → **(2)** tri par `qsort` → **(3)** `MedianInsert`.

5.2 Opérations ensemblistes

Les neuf opérations s'appuient sur trois fonctions d'assistance :

- `BSTsEqual(A,B)` : collecte les deux tableaux de mots et les compare élément par élément.
- `SentenceExistsIn(bst, liste)` : parcourt la liste et appelle `BSTsEqual` pour chaque phrase.

- `ParagraphExistsIn(phrases, liste)` : parcourt la liste et appelle `SentenceListsEqual`.

Table 4: Implémentation des opérations ensemblistes

Opération	Algorithme
<code>WordUnion</code>	Copier A dans le résultat ; copier B (l'ABR rejette les doublons)
<code>WordIntersection</code>	Collecter les mots de A ; ne garder que ceux trouvés dans B par <code>Search</code> ; <code>MedianInsert</code>
<code>WordDifference</code>	Collecter les mots de A ; ne garder que ceux <i>absents</i> de B ; <code>MedianInsert</code>
<code>SentenceUnion</code>	Copie profonde de A ; ajouter les phrases de B absentes du résultat
<code>SentenceIntersection</code>	Ajouter les phrases de A présentes dans B
<code>SentenceDifference</code>	Ajouter les phrases de A absentes de B
<code>ParagraphUnion</code>	Copie profonde de A ; ajouter les paragraphes de B absents du résultat
<code>ParagraphIntersection</code>	Ajouter les paragraphes de A présents dans B
<code>ParagraphDifference</code>	Ajouter les paragraphes de A absents de B

5.3 Gestion de la mémoire

Une couche centralisée dans `utils.c` garantit qu'aucune erreur d'allocation ne passe inaperçue :

- `CheckedMalloc` et `CheckedRealloc` terminent le processus en cas de mémoire insuffisante, plutôt que de retourner `NULL`.
- Chaque structure implémente une fonction de libération en cascade : `FreeFileList` → `FreeParagraphList` → `FreeSentenceList` → `FreeTree`.
- Tous les pointeurs sont mis à `NULL` après libération pour prévenir tout accès après libération.

6 Interface Utilisateur

6.1 Architecture visuelle

L'interface est construite autour d'un cadre centré de 100 caractères, dessiné avec des caractères Unicode. Deux modes d'affichage sont utilisés :

- **Cadre double panneau** (`DrawBox`) : le panneau gauche affiche l'art ASCII ESI / ADDS / TP ; le panneau droit contient 24 emplacements de contenu.
- **Cadre pleine largeur** (`DrawFullBox`) : un seul panneau de 98 colonnes pour l'aperçu des fichiers et l'affichage des résultats.

6.2 Fonctionnalités clés

- **Navigation par touche unique** : flèches directionnelles, raccourcis numériques et ESC, traités en mode terminal brut.
- **Interface bilingue** : la touche L bascule entre anglais et français. Toutes les chaînes sont stockées dans une table **Str** sélectionnée à l'exécution.
- **Écran de chargement persistant** : après le chargement d'un fichier, l'écran reste actif pour de nouveaux chargements.
- **Aperçu paginé** : les paragraphes s'affichent 12 par page avec les 10 premiers mots suivis de points de suspension.
- **Barre de progression animée** : panneau centré avec indicateur rotatif et remplissage en dégradé.

7 Tests et Validation

7.1 Fichiers de test

Deux fichiers de test sont fournis :

- **tests/test1.txt** : 36 paragraphes (4 groupes répétés de 9 lignes). Sollicite la pagination, la troncature de l'aperçu et les opérations sur un contenu répété.
- **tests/test2.txt** : 3 paragraphes de 3 phrases chacun. Fournit un deuxième jeu de données pour les opérations inter-fichiers.

7.2 Cas limites traités

- Fichier vide → **ParagraphList** vide, message adéquat dans l'IU.
- Paragraphe sans '.' → flushé comme une seule phrase au '\n'.
- Fichier ne finissant pas par '\n' → flush final à la fin de fichier.
- Mot entièrement constitué de ponctuation → rejeté après normalisation.
- Intersection vide → affichage de $\emptyset$.

8 Conclusion

Ce projet illustre une application cohérente et complète des structures de données classiques à un problème de traitement linguistique. L'arbre binaire de recherche apporte une sémantique ensembliste naturelle au niveau des mots ; les listes chaînées avec tableaux d'index gèrent dynamiquement les niveaux phrase et paragraphe tout en préservant un accès en $O(1)$. La stratégie d'équilibrage par insertion médiane atteint la hauteur optimale d'un ABR sans le surcoût des rotations AVL.

L'interface en mode terminal, avec son support bilingue, ses animations et son système de navigation structuré, constitue un rendu soigné qui satisfait l'ensemble des exigences du TP ADDS.